Documento de Arquitectura a Detalle del Orquestador de Nube Privada Eucalyptus

1st Wendy Cueva
Departamento de Ciencias de la Computación
Universidad de las Fuerzas Armadas ESPE
Sangolquí, Ecuador
wncueva@espe.edu.ec

2nd Rafael Vieira
Departamento de Ciencias de la Computación
Universidad de las Fuerzas Armadas ESPE
Sangolquí, Ecuador
jrvieira@espe.edu.ec

3rd Josue Zambrano
Departamento de Ciencias de la Computación
Universidad de las Fuerzas Armadas ESPE
Sangolquí, Ecuador
jazambrano21@espe.edu.ec

4th Jorge Yugsi
Departamento de Ciencias de la Computación
Universidad de las Fuerzas Armadas ESPE
Sangolquí, Ecuador
jlyugsi@espe.edu.ec

***Resumen*—Este documento describe en detalle la arquitectura de Eucalyptus, un orquestador de infraestructura de nube privada (IaaS) compatible con las API de Amazon Web Services. Se presenta su visión general, su arquitectura conceptual y lógica (Cloud Controller, User-Facing Services, Object Storage Gateway, Cluster Controller, Storage Controller y Node Controller), su arquitectura física y de despliegue, la topología de red y los puertos, los flujos de datos de procesos clave (lanzamiento de una instancia, auto-escalado y failover), la pila tecnológica y las consideraciones de seguridad. El análisis se apoya en la documentación pública del proyecto y en una instalación real de la versión 4.4.6 (FastStart) en una máquina virtual con CentOS 7.9 sobre VMware Workstation Pro, de la cual se tomaron los datos de hardware, red, servicios y problemas encontrados. Se concluye que Eucalyptus ofrece una arquitectura modular y didáctica para entender cómo funciona una nube, pero que la falta de desarrollo activo y su dependencia de software al final de su vida útil limitan su uso a laboratorio y estudio.**

Palabras clave*—*Eucalyptus, orquestación, nube privada, IaaS, Cloud-in-a-Box, KVM, compatibilidad con AWS, arquitectura de software.

I. INTRODUCCIÓN

A. Propósito del documento

Este documento es el entregable técnico principal del Tema 4 de la asignatura Tecnologías Emergentes. Su propósito es que cualquier profesional de TI pueda entender cómo funciona Eucalyptus, cómo está diseñado, de qué componentes se compone y cómo se relacionan entre sí. Para ello se describen la arquitectura conceptual y lógica, la arquitectura física y de despliegue, los flujos de datos, la pila tecnológica y los aspectos de seguridad.

El documento no sustituye a la guía oficial de instalación [3], sino que la complementa con una visión de arquitectura y con la experiencia de una instalación completa documentada por el grupo en el manual de instalación [6]. Cuando un dato proviene de esa práctica se indica de forma explícita, para distinguirlo de lo que describe la documentación pública del proyecto.

B. Visión general del orquestador

Eucalyptus (*Elastic Utility Computing Architecture for Linking Your Programs To Useful Systems*) es una plataforma de software libre que convierte un conjunto de servidores en una nube privada de tipo IaaS (Infraestructura como Servicio). Nació como proyecto de investigación en la Universidad de California en Santa Bárbara [2] y su característica distintiva es que expone las mismas API que Amazon Web Services, como EC2 para cómputo, S3 para objetos, EBS para volúmenes e IAM para identidades. Gracias a ello, herramientas pensadas para AWS pueden usarse contra una nube propia.

El problema que resuelve es el de la gestión manual de infraestructura. Crear máquinas virtuales una por una, asignarles red y disco, controlar quién puede usarlas y liberarlas cuando ya no se necesitan no escala cuando hay muchos usuarios o muchas máquinas. Un orquestador automatiza ese ciclo mediante una API: el usuario pide recursos y la plataforma decide en qué servidor se ejecutan, prepara el disco y la red, aplica las reglas de seguridad y lleva la cuenta de lo que se usa. Esto coincide con la definición de IaaS del NIST, donde el usuario aprovisiona cómputo, almacenamiento y red sin administrar el hardware subyacente [1].

Es importante ubicar el estado actual del proyecto. El repositorio principal de Eucalyptus en GitHub fue archivado en agosto de 2024 [4], de modo que ya no recibe desarrollo activo. La línea 4.4.x que instala el script FastStart solo funciona sobre CentOS o RHEL 7 [6], un sistema cuyo soporte también terminó. Por esa razón, este trabajo trata a Eucalyptus como un caso de estudio de arquitectura y como una nube de laboratorio, no como una recomendación para producción.

C. Eucalyptus y las características esenciales de la nube

El NIST describe cinco características esenciales de la computación en la nube: autoservicio bajo demanda, amplio acceso por red, agrupación de recursos, elasticidad rápida y servicio medido [1]. Eucalyptus las cubre con mecanismos concretos. El autoservicio se logra con la API y la consola, que permiten a un usuario crear sus propios recursos sin intervención del administrador. El acceso por red se da mediante los servicios web y las herramientas euca2ools. La agrupación de recursos ocurre cuando los nodos de un clúster ponen su CPU y memoria a disposición de todas las cuentas. La elasticidad la aportan el escalado automático y el balanceador de carga, y el servicio medido se apoya en el servicio de reportes y en las cuotas por cuenta, cuyos servicios aparecieron registrados en la práctica [6].

D. Alcance y metodología

El análisis combina tres fuentes: la documentación oficial y el repositorio de recetas de instalación del proyecto [3], [5], el artículo académico que presentó el sistema [2] y la instalación realizada por el grupo [6]. Esta última se hizo en una sola máquina virtual con 4 núcleos, 12 GB de RAM y 160 GB de disco sobre VMware Workstation Pro, en la configuración conocida como Cloud-in-a-Box, en la que todos los componentes conviven en el mismo servidor. De esa instalación se toman los servicios registrados, las direcciones de red, los puertos observados y los errores resueltos.

E. Conceptos clave y terminología

La plantilla habitual de este tipo de documentos menciona contenedores, pods y nodos, términos propios de Kubernetes. Eucalyptus orquesta **máquinas virtuales**, no contenedores: la unidad de trabajo es la *instancia*, equivalente a una instancia de EC2. La Tabla I resume los términos que se usan en el resto del documento y su equivalencia con otros orquestadores.

TABLA I
TERMINOLOGÍA CLAVE

Término	Significado en Eucalyptus
Orquestación	Automatizar el aprovisionamiento, la red, el almacenamiento y el ciclo de vida de los recursos mediante una API.
IaaS / nube privada	Infraestructura ofrecida como servicio dentro de la red de una organización.
Instancia	Máquina virtual en ejecución (equivale a una instancia de EC2; en Kubernetes sería la carga de trabajo, no el pod).
EMI	Eucalyptus Machine Image: plantilla de disco con la que se crea una instancia (equivale a una AMI).
Nodo	Servidor físico o virtual que ejecuta instancias (Node Controller); en Kubernetes, un worker node.
Clúster / zona	Grupo de nodos administrado por un Cluster Controller; aparece como zona de disponibilidad.
Cloud-in-a-Box	Despliegue de todos los componentes en un solo servidor, para laboratorio.
Grupo de seguridad	Conjunto de reglas de firewall que se aplica a las instancias.
Volumen (EBS)	Disco en bloque que se conecta a una instancia y persiste fuera de ella.
Puente (bridge)	Interfaz de red virtual (br0) que conecta las instancias con la red física.

F. Organización del documento

La sección II presenta la arquitectura conceptual y lógica; la sección III, la arquitectura física, los requisitos y la red; la sección IV, los flujos de datos y los diagramas de secuencia; la sección V, la pila tecnológica; la sección VI, la seguridad; y la sección VII, las conclusiones, las fortalezas y debilidades y los casos de uso ideales.

II. ARQUITECTURA CONCEPTUAL Y LÓGICA

A. Principios de diseño

Eucalyptus se organiza en tres niveles que reflejan la jerarquía de una nube: el nivel de nube, donde viven la API y el estado global; el nivel de clúster, que agrupa nodos y administra red y almacenamiento; y el nivel de nodo, donde realmente se ejecutan las máquinas virtuales [2], [3]. Cada nivel se compone de servicios independientes que se comunican por red mediante mensajes de servicios web, lo que permite instalarlos juntos en un solo servidor o repartirlos en varios.

Tres decisiones de diseño explican gran parte de la arquitectura. La primera es la compatibilidad con las API de AWS, que obliga a que los servicios tengan la misma semántica que EC2, S3, EBS e IAM. La segunda es la separación entre el plano de control (decide) y el plano de ejecución (hace), de modo que un fallo en un nodo no detiene la nube completa. La tercera es el uso de tecnologías estándar de Linux (KVM, libvirt, iptables, puentes de red) en lugar de componentes propios, lo que reduce el código a mantener.

B. Diagrama de arquitectura lógica

La Fig. 1 muestra la arquitectura lógica simplificada. Los usuarios y administradores llegan a la nube por medio de herramientas cliente o de la consola web; ambas hablan con los servicios de cara al usuario. El Cloud Controller coordina el resto de componentes y guarda el estado en una base de datos. Debajo, el Cluster Controller y el Storage Controller administran un clúster, y los Node Controller ejecutan las instancias.

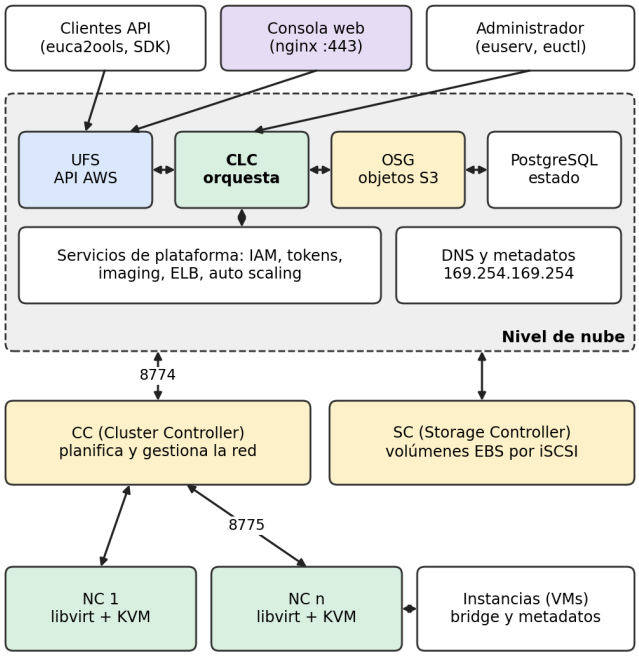


Fig. 1. Arquitectura lógica simplificada de Eucalyptus: clientes, nivel de nube, nivel de clúster y nivel de nodo.

C. Componentes del nivel de nube

Cloud Controller (CLC). Es el cerebro de la plataforma. Recibe las peticiones ya autenticadas, mantiene el inventario de recursos de toda la nube, decide a qué clúster enviar cada instancia y lleva la contabilidad de cuotas, direcciones IP y grupos de seguridad. Es la única pieza que habla con la base de datos central. Además ofrece a las instancias el servicio de metadatos, accesible en la dirección 169.254.169.254, desde el cual cada máquina virtual lee su identificador, su clave pública y sus datos de usuario.

User-Facing Services (UFS). Son la puerta de entrada de la API compatible con AWS: reciben las solicitudes HTTP o HTTPS, verifican la firma y la identidad del solicitante y las traducen en operaciones internas. Aquí se atienden EC2, IAM, STS y otros servicios de cara al usuario por el puerto 8773, que es también el que usan las herramientas euca2ools [3], [6].

Object Storage Gateway (OSG). Ofrece almacenamiento de objetos compatible con S3. Además de guardar los datos de los usuarios, aloja las imágenes de máquina (EMI) que los nodos descargan al lanzar una instancia. Por defecto utiliza un servicio de respaldo local, llamado Walrus en el registro de servicios, que guarda los objetos en el disco del servidor.

Servicios de plataforma. Sobre el núcleo se ejecutan servicios adicionales que también se registran como parte de la nube. En la instalación del grupo, el comando de listado de servicios mostró los que aparecen en la Tabla II, todos en estado *enabled* [6].

TABLA II
SERVICIOS REGISTRADOS EN LA INSTALACIÓN

Servicio	Función
compute / eucalyptus	API de cómputo y lógica central del CLC
identity, euare, tokens	Identidades, usuarios, políticas y tokens temporales (IAM, STS)
objectstorage, walrusbackend	Almacenamiento de objetos S3 y su respaldo local
storage (default-sc-0)	Volúmenes en bloque del clúster default
imaging, imagingbackend	Importación y conversión de imágenes y volúmenes
loadbalancing y su backend	Balanceador de carga elástico (ELB)
dns, notifications, reporting	DNS interno, notificaciones y reportes de uso
simpleworkflow, user-api	Flujos de trabajo y API de usuario

D. Componentes del nivel de clúster

Cluster Controller (CC). Administra un grupo de nodos. Recibe del CLC la orden de lanzar una instancia, elige el nodo que la ejecutará según una política de planificación (por defecto, de turnos rotativos o *round robin* en las recetas de instalación [5]) y reenvía las órdenes al nodo elegido. También recopila el estado y la capacidad libre de sus nodos y se lo informa al CLC. Escucha en el puerto 8774.

Storage Controller (SC). Proporciona almacenamiento en bloque, equivalente a EBS. Crea volúmenes, los publica por iSCSI y los asocia a las instancias en ejecución, además de administrar las instantáneas. Admite varios *backends*; el más simple, usado en instalaciones de laboratorio, guarda los volúmenes como archivos dentro del propio servidor, en el directorio de datos de Eucalyptus bajo */var*. Esta es la razón por la que el particionado del disco en el laboratorio debió dejar casi todo el espacio en la partición raíz [6].

E. Componentes del nivel de nodo

Node Controller (NC). Es el componente que realmente ejecuta las máquinas virtuales. Recibe órdenes del CC por el puerto 8775, descarga y prepara la imagen, construye el disco de la instancia y pide a libvirt que la arranque sobre el hipervisor KVM [7]. Informa periódicamente su estado y el de cada instancia. En la instalación del grupo, el servicio *eucalyptus-node* se ejecutó bajo el servidor web Apache httpd, como mostró el estado del servicio systemd [6].

eucanetd. Es el demonio de red. Traduce los grupos de seguridad, las direcciones IP públicas y privadas y el acceso al servicio de metadatos en reglas de iptables, ipsets y puentes de red en cada nodo, de manera que las instancias queden aisladas según la política definida por el usuario. El manual de la práctica indica que Eucalyptus administra sus

propias reglas de red mediante este componente, por lo que no hace falta abrir puertos a mano [6].

F. Consola web y herramientas cliente

La **Eucalyptus Console** es una aplicación web escrita en Python que permite administrar instancias, volúmenes, imágenes y grupos de seguridad desde un navegador. En la instalación del grupo se publica por HTTPS en el puerto 443 mediante nginx, y el servicio interno escucha en el puerto 8888 [6]. Las herramientas de línea de comandos **euca2ools** ofrecen comandos con prefijo *euca-* para el uso normal y *euserv-* y *euctl* para la administración de servicios y propiedades de la nube.

G. Estructura interna y rutas de trabajo

Aunque el CLC parece un único proceso, en realidad aloja varios servicios lógicos que comparten la máquina virtual de Java: el servicio de cómputo, el de identidad, el de almacenamiento de objetos, el de balanceo, el de DNS y otros, cada uno con su propio nombre registrado. Esto explica por qué el listado de servicios de la Tabla II muestra muchas entradas aunque el servidor físico sea uno solo. Cuando se instala en servidores distintos, cada servicio puede asignarse a un host diferente sin cambiar su interfaz.

En el nodo, el comportamiento se controla con el archivo de configuración */etc/eucalyptus/eucalyptus.conf*, compartido por el CC y el NC. Entre sus variables se encuentran el usuario del sistema, los puertos del CC y del NC, la ruta donde se guardan los discos de las instancias y la interfaz de red que se usa como puente. Un extracto típico es el siguiente [5].

```
EUCA_USER="eucalyptus"
CC_PORT="8774"
NC_PORT="8775"
INSTANCE_PATH="/var/lib/eucalyptus/instances"
```

El directorio de instancias contiene dos áreas que conviene dimensionar: una caché donde se guardan las imágenes ya descargadas, para que lanzar una segunda instancia con la misma EMI no repita la descarga, y un área de trabajo donde se construye el disco de cada instancia. Los volúmenes del SC se guardan en */var/lib/eucalyptus/volumes* y los objetos del servicio local de almacenamiento, en un directorio propio del mismo árbol. Todo esto vive bajo */var*, lo que confirma la importancia del particionado explicado en la sección III.

H. Equivalencia con los servicios de AWS

Como Eucalyptus busca la compatibilidad con AWS, cada componente tiene un equivalente conocido, lo que ayuda a quien ya trabaja con nubes públicas a entender la arquitectura. El CLC y el UFS reemplazan al plano de control y a los puntos de acceso de EC2; el OSG, a S3; el SC, a EBS; los NC, a los servidores físicos que ejecutan las instancias; el servicio de identidad, a IAM y STS; el balanceador, a ELB; y el servicio de métricas y alarmas, a CloudWatch. La diferencia está en la escala: mientras en una nube pública estos servicios se reparten entre miles de servidores, en Eucalyptus pueden convivir en uno solo.

I. Interacciones y supervisión entre componentes

La comunicación entre niveles es de tipo petición y respuesta, pero la supervisión es periódica. El CC consulta de forma regular a cada NC mediante una operación de descripción de recursos para conocer su capacidad libre y el estado de sus instancias; si un NC deja de recibir o responder esas consultas durante un tiempo, el sistema de monitoreo lo advierte como alerta o como error según el plazo transcurrido [4]. Del mismo modo, el CC mantiene una caché de imágenes propia, en */var/lib/eucalyptus/dynserv*, cuyo tamaño conviene vigilar junto con el de los volúmenes y los directorios de instancias [4].

Este modelo de consulta periódica tiene una consecuencia de diseño: el estado que ve el usuario nunca es instantáneo, sino el último reporte recibido. Por eso, tras lanzar una instancia, la consulta puede mostrar *pending* durante algunos segundos aunque la máquina ya esté arrancando. El diseño privilegia la simplicidad y la tolerancia a fallos parciales frente a la sincronía estricta.

J. Alta disponibilidad por componente

Según el modelo descrito en la documentación, los componentes de control pueden desplegarse en parejas para tolerar fallos: el CLC, el UFS y el OSG en el nivel de nube, y el CC y el SC en el nivel de clúster. En cada pareja, una réplica atiende y la otra espera. Los NC, en cambio, escalan hacia afuera: si un nodo falla, las instancias que corrían en él se pierden y deben relanzarse en otro nodo, pero el resto de la nube sigue funcionando. Esta diferencia es importante al diseñar una instalación: la redundancia se invierte en el plano de control, y la tolerancia a fallos de las aplicaciones se resuelve con varias instancias detrás de un balanceador.

K. Registro y estados de los servicios

Cada servicio se registra en el CLC con un nombre, un tipo y una dirección, y mantiene un estado. Los estados principales son *ENABLED* (activo y atendiendo), *DISABLED* (listo pero en espera, típico de una réplica), *NOTREADY* y *BROKEN* (con fallas) y *STOPPED*. El comando de listado de servicios muestra estos estados y fue la herramienta principal de verificación posterior a la instalación [6]. Este mecanismo es la base de la alta disponibilidad que se describe en la sección IV.

III. ARQUITECTURA FÍSICA Y DE DESPLIEGUE

A. Topologías de despliegue

Eucalyptus admite dos formas de reparto físico. En una instalación distribuida, el CLC, el UFS y el OSG se colocan en uno o más servidores de gestión, el CC y el SC en servidores de clúster y los NC en tantos servidores de cómputo como se necesiten; así la capacidad crece añadiendo nodos. En la instalación *Cloud-in-a-Box*, todos los componentes se instalan en un único servidor mediante el script FastStart, lo que simplifica el despliegue a costa de la escala [3], [6]. El grupo utilizó esta segunda opción dentro de una máquina virtual con virtualización anidada.

TABLA III
TOPOLOGÍAS DE DESPLIEGUE

Aspecto	**Cloud-in-a-Box**	**Distribuida**
Servidores	1	Varios (gestión, clúster, nodos)
Escalabilidad	Limitada a una máquina	Se añaden nodos NC
Alta disponibilidad	No	Posible con réplicas del CLC/UFS
Instalación	FastStart automático	Paquetes y registro manual
Uso típico	Laboratorio y enseñanza	Nube de equipo o de campus

La principal diferencia práctica está en el registro de servicios. En una instalación distribuida cada componente debe registrarse en el CLC con su dirección, mientras que FastStart lo hace de forma automática en el mismo host. Para el estudio de la arquitectura, ambas formas son equivalentes en lo lógico: cambia solo en qué máquina se ejecuta cada servicio.

B. Diagrama de despliegue

La Fig. 2 muestra cómo se distribuyen los componentes en la práctica. El host Windows ejecuta VMware Workstation Pro, que aloja la máquina virtual *Eucalyptus-Grupo3*. Dentro de ella conviven el CLC, UFS, OSG y SC en un único proceso Java, la base de datos, el CC, el NC con KVM, eucanetd y la consola web. Las instancias de usuario son, a su vez, máquinas virtuales anidadas, por eso es obligatorio activar la virtualización anidada (VT-x o AMD-V) en la VM [6].

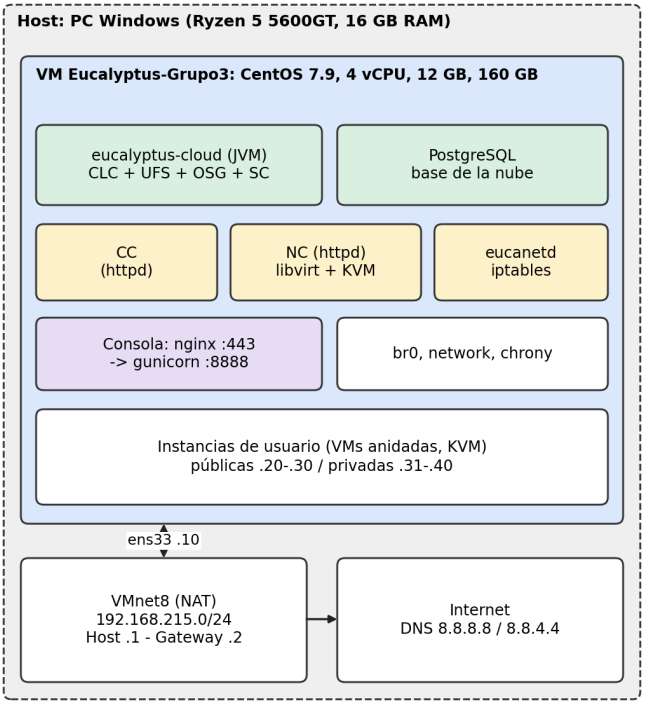


Fig. 2. Diagrama de despliegue de la instalación Cloud-in-a-Box sobre VMware Workstation Pro.

C. Requisitos de hardware

La Tabla IV compara los requisitos mínimos y recomendados con los valores usados en el laboratorio. El equipo físico tenía 16 GB de RAM, de modo que se asignaron 12 GB a la VM y se dejaron 4 GB al sistema anfitrión. El manual advierte que no conviene bajar de 8 GB, porque el instalador puede negarse a continuar [6].

TABLA IV
REQUISITOS DE HARDWARE

Recurso	**Mínimo**	**Recomendado**	**Usado**
CPU	4 núcleos con VT-x/AMD-V	8 núcleos	4 núcleos (1x4)
RAM	8 GB	16 GB o más	12 GB
Disco	100 GB	160 GB o más	160 GB, archivo único
Red	1 interfaz	1 interfaz	1 NAT (VMnet8)
Virtualización	Activa en BIOS	Activa en BIOS	VT-x/AMD-V + anidada

D. Requisitos de software

La línea 4.4 de Eucalyptus exige CentOS o RHEL 7 de 64 bits [3]. En la práctica se usó CentOS 7.9.2009 en su edición Minimal. Como el soporte de CentOS 7 terminó, sus repositorios deben apuntar al archivo histórico (vault) [11], y los repositorios de EPEL 7 se redirigen al archivo de Fedora [10]. La Tabla V resume el software empleado.

TABLA V
SOFTWARE UTILIZADO EN LA PRÁCTICA

Software	Versión / detalle
Sistema operativo	CentOS 7.9 Minimal (x86_64)
Virtualizador	VMware Workstation Pro 17
Eucalyptus	4.4.6 (FastStart)
Reemplazo de Chef	Cinc Client 15.17.4 [9]
Hipervisor	KVM mediante libvirt
Servicios del sistema	chrony, network, nginx, httpd
Cliente	Navegador, PowerShell con SSH

E. Validación de la arquitectura desplegada

Una vez instalada la nube, la arquitectura puede validarse con pocos comandos, que además permiten confirmar que cada nivel descrito en la sección II existe y está activo. El primero lista los servicios registrados; el segundo muestra la zona de disponibilidad con su clúster y los tipos de instancia; y el tercero lista las imágenes.

```
eval $(clcadmin-assume-system-credentials)
euserv-describe-services
euca-describe-availability-zones verbose
euca-describe-images
```

En la práctica, el primer comando mostró el servicio de almacenamiento asociado al clúster *default* con el nombre *default-sc-0* y el servicio de respaldo de objetos con el nombre *walrus-0*; el segundo mostró la zona *default* con su clúster *default-cc-0*; y el tercero listó dos imágenes en estado *available* [6]. Estos resultados confirman la presencia del CLC, el CC, el SC y el OSG, y dejan la nube lista para lanzar una primera instancia.

F. Servicios del sistema y orden de arranque

En el servidor, los componentes se gestionan como servicios de systemd. El servicio *eucalyptus-cloud* ejecuta el CLC, el UFS, el OSG y el SC dentro de procesos Java; el CC dispone de su propio servicio de clúster; y el servicio *eucalyptus-node* ejecuta el NC bajo Apache httpd. En la práctica, las dos consultas de estado mostraron ambos servicios como *active (running)* y el primero con dos procesos Java asociados a la dirección 192.168.215.10 [6]. El orden de arranque importa: primero deben estar activos la red y el reloj, luego el servicio de nube, y después los servicios de clúster y de nodo, que se registran contra el CLC.

```
systemctl status eucalyptus-cloud --no-pager | head -n 8
systemctl status eucalyptus-node --no-pager | head -n 8
```

G. Almacenamiento y particionado

Eucalyptus guarda las imágenes en caché, los discos de las instancias y los volúmenes dentro de */var*. Con el esquema por defecto de CentOS, que reserva unos 50 GB para la raíz y el resto para */home*, el instalador se queda sin espacio. Por ello el grupo eliminó */home* y asignó a la raíz todo el espacio disponible con LVM, de modo que la partición / quedó con unos 153 GiB, además de */boot* de 1 GiB y un área de intercambio cercana a 6 GiB [6]. Esta es una decisión de arquitectura física que influye de forma directa en la capacidad de la nube.

H. Comprobaciones de red previas a la instalación

Antes de lanzar el instalador, el manual de la práctica propone una serie de comprobaciones que equivalen a validar la topología física descrita arriba. Con *ip a* se confirma que la interfaz *ens33* tiene la dirección 192.168.215.10/24. Con *ping* a 8.8.8.8 se comprueba la salida hacia Internet y, con *ping* a *vault.centos.org*, la resolución de nombres. Con *chronyc sources* se verifica que haya servidores de hora alcanzables, y con la búsqueda de las banderas *vmx* o *svm* en */proc/cpuinfo* se confirma que la virtualización anidada está activa [6].

Si cualquiera de estas pruebas falla, la instalación fallará más adelante con mensajes menos claros. Por eso conviene tratarlas como requisitos de la arquitectura y no como pasos opcionales, y repetirlas después de cualquier cambio de red, como el que realiza Eucalyptus al crear el puente *br0*.

I. Capacidad y tipos de instancia

Eucalyptus define tipos de instancia que fijan los núcleos, la memoria y el disco de cada máquina virtual, de forma análoga a los tipos de EC2. Al listar la zona de disponibilidad de la práctica, el sistema mostró para cada tipo cuántas instancias caben en la capacidad libre y cuántas en total. Con 4 núcleos, la nube podía alojar como máximo 4 instancias de un núcleo, y solo una instancia de los tipos de 4 núcleos [6]. La Tabla VI resume algunos de los tipos observados.

TABLA VI
TIPOS DE INSTANCIA OBSERVADOS

Tipo	vCPU	RAM (MB)	Disco (GB)
m1.small	1	256	5
m1.medium	1	512	10
m1.large	2	512	10
m1.xlarge	2	1024	10
c1.xlarge	2	2048	10
m3.xlarge	4	2048	15

Esta tabla ilustra un principio del dimensionamiento: la capacidad de la nube la limita el recurso más escaso del nodo. En la VM del laboratorio el límite fueron los núcleos y la memoria, por lo que no se pueden lanzar muchas instancias grandes a la vez. Para una nube real habría que sumar los recursos de todos los nodos y dejar margen para los servicios propios de Eucalyptus.

J. Consideraciones de dimensionamiento y crecimiento

El número de instancias que admite una nube de Eucalyptus se calcula, para cada tipo, como el menor cociente entre los recursos libres de los nodos y los que pide el tipo: núcleos, memoria y disco. Con un nodo de 4 núcleos y 12 GB, un tipo de 1 núcleo y 512 MB estaría limitado por los núcleos (4 instancias), mientras que un tipo de 2 núcleos y 2 GB admitiría 2 instancias por el mismo motivo. Esa regla explica la tabla de capacidad que mostró el sistema en la práctica [6].

Para crecer, la arquitectura permite añadir nodos NC al clúster, añadir clústeres con su propio CC y SC, y separar el CLC en un servidor dedicado. El crecimiento tiene un límite natural en el CLC, que es único para la nube y concentra la base de datos; por eso la alta disponibilidad de ese componente es la que más condiciona el tamaño máximo de una instalación.

K. Topología de red

La práctica usó la red NAT de VMware (VMnet8) con la subred 192.168.215.0/24. El host Windows tiene la dirección .1, la puerta de enlace NAT es la .2 y el rango DHCP de VMware empieza en la .128, de modo que las direcciones usadas por Eucalyptus no chocan con las de VMware. El servidor tiene la dirección fija 192.168.215.10, y el rango 192.168.215.20 a .40 se reservó para las instancias: el instalador lo dividió en direcciones públicas (.20 a .30) y privadas (.31 a .40) [6]. La Fig. 3 muestra esta topología y la Tabla VII resume el plan de direcciones.

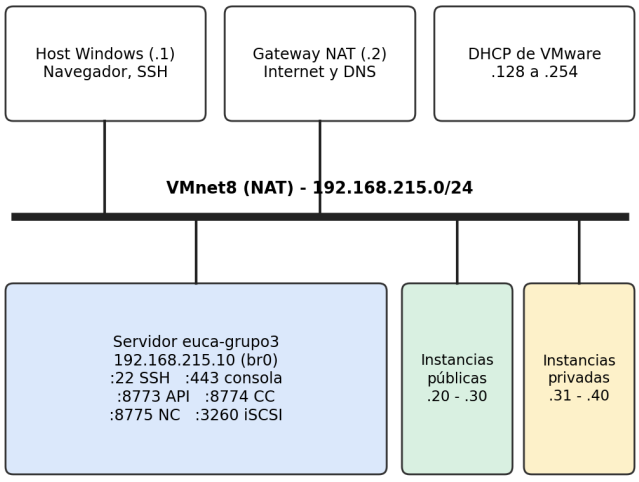


Fig. 3. Topología de red de la instalación: host, gateway NAT, servidor Eucalyptus e instancias.

TABLA VII
PLAN DE DIRECCIONES IP

Elemento	Valor
Subred VMnet8	192.168.215.0/24
Host Windows	192.168.215.1
Gateway NAT	192.168.215.2
Servidor (euca-grupo3)	192.168.215.10
IPs públicas de instancias	192.168.215.20 a .30
IPs privadas de instancias	192.168.215.31 a .40
DHCP de VMware	192.168.215.128 a .254
DNS	8.8.8.8 y 8.8.4.4

Durante la fase 2 de la instalación, Eucalyptus crea un puente de red *br0* y mueve hacia él la dirección IP que tenía la interfaz *ens33*; para que esto funcione, el servicio *network* debe reemplazar a NetworkManager. En esa etapa, el archivo de resolución de nombres quedó sin servidores DNS y fue necesario escribirlos a mano [6]. Los modos de red de Eucalyptus determinan cómo se asignan las direcciones y cómo se aíslan las instancias; en la serie 4.x se documentan el modo *EDGE*, donde el aislamiento depende de reglas en cada nodo, y el modo *VPCMIDO*, que crea redes virtuales tipo VPC mediante MidoNet [3]. La Tabla VIII compara ambos modos y la Tabla IX lista los puertos que intervienen.

TABLA VIII
MODOS DE RED EN LA SERIE 4.X

Aspecto	EDGE	VPCMIDO
Modelo	Las instancias se conectan a un puente del nodo	Redes virtuales tipo VPC sobre MidoNet
Aislamiento	Reglas de iptables y ipsets por nodo	Redes aisladas por cuenta y por VPC
Complejidad	Baja, ideal para laboratorio	Alta, requiere componentes adicionales
Direcciones públicas	Rango asignado por el administrador	Gestión mediante gateways virtuales

TABLA IX
PUERTOS PRINCIPALES

Puerto	Servicio	Uso
22/TCP	SSH	Administración remota del servidor
443/TCP	nginx (consola)	Consola web por HTTPS
8888/TCP	Consola (interno)	Servicio detrás de nginx
8773/TCP	CLC / UFS / OSG / SC	API de Eucalyptus y euca2ools
8774/TCP	Cluster Controller	Órdenes del CLC al clúster
8775/TCP	Node Controller	Órdenes del CC al nodo
3260/TCP	iSCSI	Volúmenes EBS hacia los nodos
80/TCP	HTTP	Servidor web de las instancias
53/UDP	DNS	Resolución de nombres

El flujo de comunicación sigue la jerarquía lógica: los clientes llegan al puerto 8773 del UFS, el CLC se comunica con el CC por el 8774, el CC con cada NC por el 8775, y los nodos descargan las imágenes desde el OSG por el 8773. Los volúmenes viajan desde el SC hasta el nodo por iSCSI, y las instancias consultan el servicio de metadatos en 169.254.169.254, dirección que eucanetd redirige hacia el CLC.

IV. FLUJO DE DATOS Y PROCESOS

A. Visión de conjunto de los flujos

Los procesos que se describen en esta sección comparten una estructura común: una solicitud entra por el UFS, el CLC la autoriza y la traduce en una orden, el CC la reparte al nodo adecuado y el NC la ejecuta, y el resultado vuelve hacia arriba como un cambio de estado. Entender este patrón permite predecir el comportamiento de cualquier otra operación de la API, como detener una instancia, crear una instantánea o liberar una dirección IP.

B. Ciclo de vida de una solicitud

Las Figs. 4 y 5 muestran, en dos partes, el recorrido de una solicitud para lanzar una instancia, desde que el usuario la envía hasta que está en ejecución. El proceso se puede describir en cuatro etapas.

Recepción y autenticación. El cliente envía la operación *RunInstances* firmada con sus claves de acceso al puerto 8773. El UFS verifica la firma y el servicio de identidad comprueba, con las políticas IAM de la cuenta, que el usuario tiene permiso para lanzar instancias.

Planificación. El CLC valida las cuotas, reserva una dirección IP y asocia los grupos de seguridad. Después decide el clúster y envía la orden al CC, que elige un nodo con capacidad libre. En ese momento el CLC ya puede responder al usuario con el identificador de la instancia y el estado *pending*.

Preparación en el nodo. El NC descarga el manifiesto y las partes de la EMI desde el OSG, las guarda en una caché local, construye el disco de la instancia y define el dominio en libvirt. Mientras tanto, eucanetd aplica las reglas de red y los grupos de seguridad.

Ejecución y reporte. KVM arranca la máquina virtual. El NC informa el estado *running* al CC, el CC al CLC, y el usuario lo ve al consultar la instancia. La instancia obtiene sus datos de arranque del servicio de metadatos.

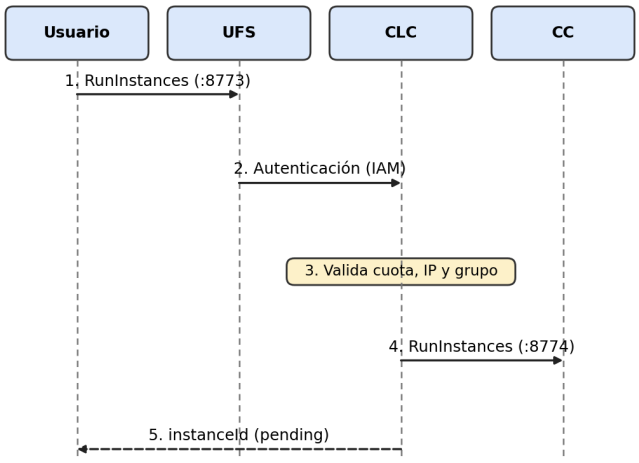


Fig. 4. Lanzamiento de una instancia (a): recepción, autenticación y planificación.

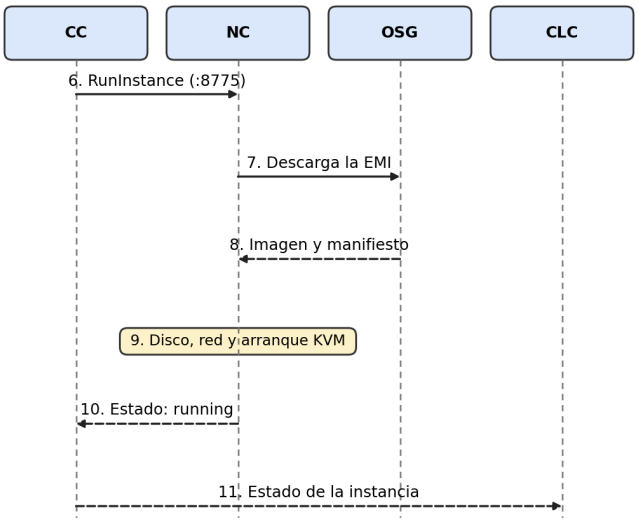


Fig. 5. Lanzamiento de una instancia (b): preparación y ejecución en el nodo.

C. Manejo de fallos durante el lanzamiento

Un lanzamiento puede fallar en varios puntos: la autenticación puede ser rechazada, la cuota puede estar agotada, el clúster puede no tener capacidad o el nodo puede no lograr descargar la imagen o iniciar el hipervisor. De forma conceptual, los dos primeros fallos se informan al usuario de inmediato en la respuesta de la API; los otros ocurren después de que el usuario ya recibió el identificador de la instancia, por lo que se reflejan como un cambio de estado (por ejemplo, la instancia pasa a *terminated* y se liberan sus direcciones y su cupo). Este comportamiento es consecuencia directa de que el lanzamiento sea asíncrono, como muestran las Figs. 4 y 5, y es el mismo que presenta EC2.

D. Terminación de una instancia

El proceso inverso sigue el mismo camino jerárquico. El usuario envía *TerminateInstances*, el UFS autentica y autoriza, el CLC ubica la instancia y pide al CC que la termine, y el CC lo reenvía al NC correspondiente. El NC detiene el dominio en libvirt, elimina el disco de trabajo de la instancia, libera sus reglas de red y reporta el nuevo estado. Los volúmenes conectados no se eliminan: pasan a estado disponible y pueden conectarse a otra instancia. El CLC, por último, devuelve a la reserva la dirección IP y el cupo de recursos que ocupaba la instancia.

E. Estados de una instancia

Una instancia pasa por los estados *pending* (se está preparando), *running* (en ejecución), *shutting-down* (apagándose) y *terminated* (eliminada). Las instancias respaldadas por volumen pueden detenerse (*stopping* y *stopped*) y volver a iniciarse sin perder su disco. Cada transición la origina una operación de la API y la ejecuta el NC, que reporta el resultado hacia arriba.

F. Auto-escalado

El escalado automático se apoya en tres servicios que imitan a los de AWS: CloudWatch para métricas y alarmas, Auto Scaling para las políticas de escalado y el balanceador de carga elástico (ELB) para repartir el tráfico. La Fig. 6 muestra el proceso: los nodos envían métricas de las instancias, una alarma se activa cuando una métrica supera un umbral durante varios periodos, la política asociada indica a Auto Scaling que aumente la capacidad, y este solicita nuevas instancias al CLC siguiendo el mismo flujo de las Figs. 4 y 5. Finalmente, la nueva instancia se registra en el balanceador cuando supera la verificación de salud. Este flujo se describe con base en el diseño de compatibilidad con AWS y no fue probado en la práctica del grupo.

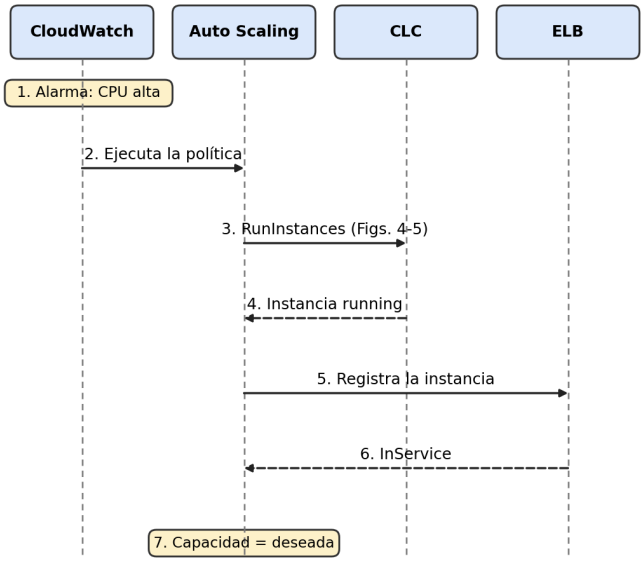


Fig. 6. Diagrama de secuencia del auto-escalado con CloudWatch, Auto Scaling y ELB.

G. Alta disponibilidad y failover

Eucalyptus aplica la alta disponibilidad registrando los servicios críticos en parejas: una réplica activa en estado *ENABLED* y otra en espera en estado *DISABLED*. El estado de la nube se replica hacia la réplica en espera. Cuando el mecanismo de gestión de topología detecta que el servicio activo deja de estar disponible (*NOTREADY*), promueve a la réplica a *ENABLED* y las nuevas solicitudes se atienden desde ella, como ilustra la Fig. 7. En la instalación Cloud-in-a-Box no hay réplicas, por lo que ese mecanismo no se pudo comprobar; el diagrama es conceptual y se basa en el modelo de estados descrito en la sección II.

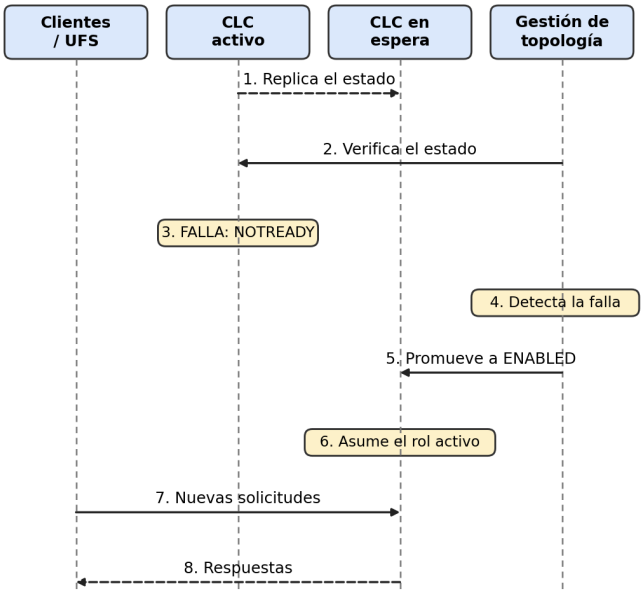


Fig. 7. Diagrama de secuencia conceptual de failover entre un CLC activo y uno en espera.

H. Registro y uso de imágenes

Una EMI se prepara fuera de la nube, se empaqueta en partes con un manifiesto y se sube al OSG, donde queda registrada y asociada a una cuenta. La instalación del grupo incluyó dos imágenes en estado *available*: una imagen pública de ejemplo, que se usará para lanzar la instancia web, y la imagen interna del servicio de balanceo e imágenes, de unos 350 MB, que el instalador descarga durante la fase 2 [6]. Cuando se lanza una instancia, el NC solo descarga la imagen si no la tiene en su caché; por eso la primera instancia de una EMI tarda más que las siguientes.

I. Conexión de un volumen

La Fig. 8 muestra cómo se crea un volumen y se conecta a una instancia. El CLC delega en el SC la creación del volumen, que queda disponible en la zona. Al pedir la conexión, el SC lo exporta por iSCSI y el NC abre una sesión hacia él por el puerto 3260 y lo presenta a la máquina virtual como un disco adicional. Mientras el volumen esté conectado, su estado es *in-use*.

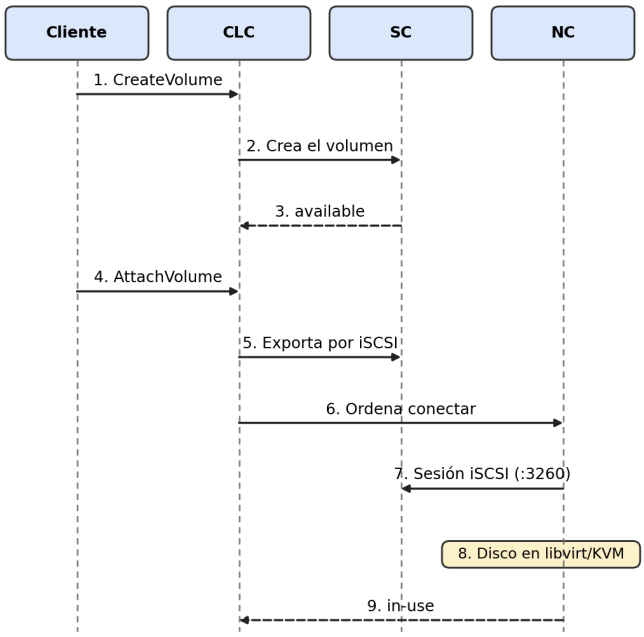


Fig. 8. Diagrama de secuencia de creación y conexión de un volumen EBS.

J. Balanceo de carga

El balanceador de carga elástico se implementa como un servicio de plataforma. Al crear un balanceador, el CLC lanza instancias especiales construidas a partir de la imagen interna del servicio, que reparten las conexiones entre las instancias registradas y verifican su salud con pruebas periódicas. Esta es la razón por la que el instalador FastStart pregunta si se desea instalar el balanceador y la imagen de servicio, y por la que esa opción añade entre 10 y 15 minutos al proceso [6].

K. Operación, registros y monitoreo

Cada componente deja sus registros en */var/log/eucalyptus*, y la consola escribe en */var/log/eucaconsole.log*, el archivo que permitió diagnosticar el error de inicio de sesión en la práctica [6]. Para el monitoreo básico basta comprobar que los puertos 8773, 8774 y 8775 estén escuchando, que el estado de los servicios sea *enabled* y que el espacio en */var* no se agote, ya que allí residen las imágenes, las instancias y los volúmenes. Además, el NC avisa al CC de forma periódica; si deja de hacerlo, el CC lo marca como no disponible y deja de planificar instancias en él.

L. Proceso de instalación con FastStart

FastStart automatiza la instalación en fases: verificación del sistema, instalación del Cloud Controller, instalación del resto de componentes y configuración final, con las recetas de Chef que el propio script descarga [3], [5]. La Fig. 9 resume las etapas y los problemas que el grupo resolvió en cada una. Chef dejó de permitir la descarga libre de versiones antiguas (error HTTP 402) y se reemplazó por Cinc 15 [9]; el repositorio EPEL 7 respondía con error 404 y se apuntó al archivo de Fedora [10]; las recetas usaban la instrucción *node.set*, que Chef 15 eliminó, y se cambió por *node.normal*; y la pérdida de los servidores DNS al crearse el puente *br0* obligó a escribirlos a mano. Por último, el inicio de sesión de la consola falló por una incompatibilidad entre su módulo *eventlet* y Python 2.7.5, y se resolvió con tres ajustes [6].

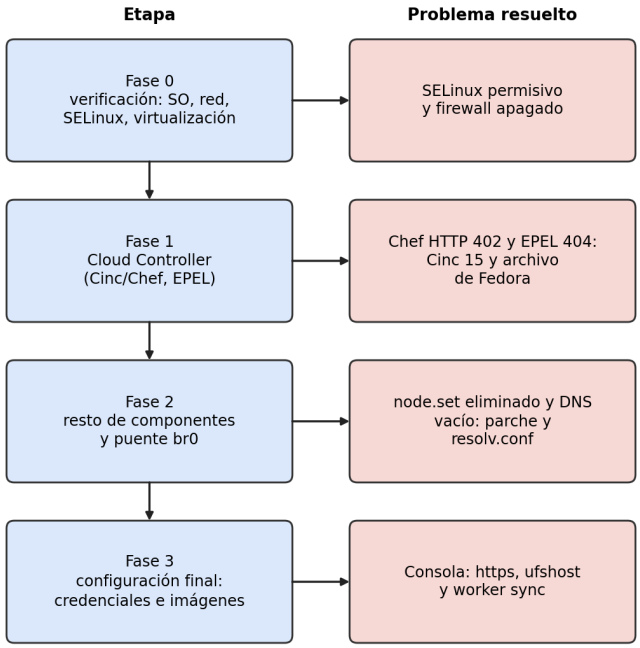


Fig. 9. Etapas de la instalación FastStart y problemas resueltos en la práctica.

M. Flujo de almacenamiento

Cuando un usuario crea un volumen, el CLC pide al SC que reserve el espacio. Al conectarlo a una instancia, el SC lo exporta por iSCSI y el NC del nodo donde corre la instancia lo vincula al dominio libvirt como un disco adicional. Las instantáneas se guardan como objetos en el OSG. El almacenamiento de objetos sigue otro camino: los clientes S3 hablan con el OSG, que escribe los datos en su *backend* y los mantiene disponibles para los nodos.

V. PILA TECNOLÓGICA

A. Lenguajes, frameworks y tecnologías base

Eucalyptus es un sistema políglota. Los servicios del nivel de nube se ejecutan sobre la máquina virtual de Java, mientras que los componentes más cercanos al hardware, el CC y el NC, están escritos en C y se publican como servicios web dentro del servidor Apache httpd, hecho que se observó en el estado del servicio del nodo [6]. La consola es una aplicación Python, y la instalación y configuración se automatizan con recetas de Chef escritas en Ruby [5]. La Tabla X resume esta pila.

TABLA X
PILA TECNOLÓGICA DE EUCALYPTUS

Capa	Tecnología
Nivel de nube (CLC, UFS, OSG, SC)	Java sobre JVM, servicio eucalyptus-cloud
Nivel de clúster y nodo (CC, NC)	C, sobre Apache httpd
Virtualización	KVM y libvirt [7]
Red de instancias	Puentes de Linux, iptables, ipsets
Consola web	Python 2.7, nginx, gunicorn
Clientes	euca2ools (Python), API compatible con AWS
Instalación	Chef/Cinc, recetas en Ruby, yum
Base de datos	PostgreSQL

B. Bases de datos, mensajería y dependencias

El estado de la nube (cuentas, usuarios, políticas, metadatos de imágenes, volúmenes e instancias) se guarda en bases de datos PostgreSQL gestionadas por el CLC. No existe un bus de mensajería externo: los componentes se comunican directamente por servicios web firmados y por los reportes periódicos del NC hacia el CC y del CC hacia el CLC. Entre las dependencias del sistema operativo están libvirt, KVM, bridge-utils, iptables, ipsets, el servicio de sincronización de hora y el soporte iSCSI para volúmenes.

La sincronización de la hora es un requisito funcional, no opcional: el manual de la práctica señala que Eucalyptus exige que el servidor tenga la hora correcta y sincronizada, y para ello configuró chrony [6]. Esto se explica porque la autenticación por firmas y los tokens temporales dependen de marcas de tiempo.

TABLA XI
DEPENDENCIAS DEL SISTEMA

Dependencia	Papel
libvirt y KVM	Definir y ejecutar las máquinas virtuales
bridge-utils	Puentes de red para las instancias
iptables / ipset	Reglas de los grupos de seguridad
Apache httpd	Aloja los servicios del CC y del NC
chrony	Hora sincronizada (firmas y tokens)
Cliente/servidor iSCSI	Volúmenes entre SC y NC
PostgreSQL	Persistencia del estado de la nube

C. Interfaces y protocolos

Los componentes de Eucalyptus se comunican con protocolos distintos según el nivel. Hacia los usuarios, la API se expone por HTTP o HTTPS con solicitudes firmadas, igual que las de AWS. Entre el CLC, el CC y el NC se intercambian mensajes de servicios web, que en el caso del CC y del NC son atendidos por Apache httpd, como se observó en el servicio de nodo [6]. Entre el NC y el hipervisor se usa libvirt de forma local, y entre el SC y el NC los volúmenes viajan por iSCSI. Las instancias, por su parte, leen sus metadatos por HTTP desde la dirección 169.254.169.254, que eucanetd redirige hacia el CLC.

Esta variedad de protocolos tiene una ventaja y un costo. La ventaja es que cada tramo usa una tecnología madura y bien conocida. El costo es que hay que conocer y proteger varios canales distintos, cada uno con su propio puerto y sus propias credenciales, lo que complica la configuración del firewall y la solución de problemas.

D. Herramientas de operación

La operación diaria se hace con euca2ools. En la práctica se usaron, entre otros, los comandos de la Tabla XII. Las credenciales de sistema se cargan con *clcadmin-assume-system-credentials*, y la configuración del administrador queda en el directorio */root/.euca* [6].

TABLA XII
COMANDOS DE VERIFICACIÓN UTILIZADOS

Comando	Propósito
euserv-describe-services	Servicios y su estado
euca-describe-availability-zones verbose	Zona, clúster y tipos de instancia
euca-describe-images	Imágenes registradas
systemctl status eucalyptus-cloud	Estado del servicio de nube
systemctl status eucalyptus-node	Estado del servicio de nodo

E. Relación con la serie 5

El proyecto publicó también una guía de instalación de la serie 5 [8], con paquetes y repositorios distintos a los de la línea 4.4 que usa FastStart. Este trabajo no instaló esa serie, de modo que no se afirma cuánto cambian su arquitectura o su pila tecnológica; se deja como trabajo futuro la comparación entre ambas líneas, ya que podría eliminar parte de las dependencias obsoletas descritas aquí.

F. Evidencia de versiones en la práctica

La instalación del grupo deja un registro útil de las versiones que componen esta pila. El kernel de CentOS fue de la serie 3.10.0-1160, la versión de Python de la consola fue 2.7.5, el reemplazo de Chef fue Cinc Client 15.17.4 y la imagen interna del servicio de balanceo e imágenes correspondió a la versión 4.4.102 del paquete de Eucalyptus [6]. Conocer estas versiones es importante para reproducir el entorno, porque un cambio en cualquiera de ellas, sobre todo en Chef o en las recetas, puede romper el instalador.

G. Riesgo por obsolescencia

La pila de Eucalyptus 4.4 depende de componentes que han llegado al final de su vida útil: CentOS 7, Python 2.7, Chef 15 y repositorios archivados. La instalación del grupo requirió cuatro parches para completarse [6]. Esto no afecta la validez de su arquitectura como material de estudio, pero sí impide considerarlo una plataforma mantenible.

VI. CONSIDERACIONES DE SEGURIDAD

A. Autenticación y autorización

Eucalyptus reproduce el modelo de identidades de AWS. Las identidades se organizan en *cuentas*; cada cuenta tiene *usuarios*, *grupos* y *roles*, y los permisos se otorgan mediante políticas IAM en formato JSON. Es decir, el control de acceso es de tipo RBAC: los permisos se asignan a grupos y roles, y los usuarios los heredan. Las peticiones a la API se autentican con claves de acceso y firma de cada solicitud, o con certificados X.509 y tokens temporales emitidos por el servicio STS. En la instalación del grupo, el acceso a la consola se hizo con la cuenta *eucalyptus* y el usuario *admin* [6]. El siguiente ejemplo muestra una política que solo permite consultar instancias.

```
{ "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["ec2:DescribeInstances"],
    "Resource": "*" }] }
```

B. Gestión de secretos

Eucalyptus no incluye un gestor de secretos dedicado. Los secretos que maneja son de cuatro clases: las claves de acceso y los certificados de cada usuario, las claves SSH de las instancias (el sistema guarda la clave pública y entrega la privada una sola vez al crearla), las credenciales de sistema del propio servidor y los certificados TLS del servicio. Las credenciales del administrador quedaron en */root/.euca* [6], por lo que el acceso a la cuenta root del servidor equivale a controlar la nube. La consola usó un certificado autofirmado, lo que obliga al navegador a mostrar una advertencia y es una limitación para entornos reales.

El resultado de la práctica deja una advertencia: el instalador crea un administrador con contraseña inicial predecible, que el propio mensaje final muestra en pantalla [6]. Debe cambiarse de inmediato, y las claves no deben compartirse en repositorios ni documentos.

C. Cuentas, cuotas y separación de recursos

La unidad de aislamiento administrativo es la cuenta. Cada cuenta ve solamente sus propios recursos: instancias, volúmenes, imágenes y grupos de seguridad. La cuenta llamada *eucalyptus* es la cuenta del administrador de la nube, y su usuario *admin* fue el que se usó para entrar a la consola en la práctica [6]. Dentro de cada cuenta, un administrador puede crear usuarios y grupos con permisos limitados, de modo que el trabajo diario no requiera el acceso total.

Las cuotas permiten limitar cuántos recursos puede consumir una cuenta, de forma que una sola cuenta no agote la capacidad de la nube. Esto se relaciona con la característica de servicio medido del NIST [1]: lo que se mide puede limitarse y facturarse internamente. En un laboratorio con pocos usuarios las cuotas son menos importantes, pero en una nube compartida son la primera defensa contra el uso excesivo accidental.

D. Ejemplo de flujo de autorización

Siguiendo el modelo de AWS, la autorización de una solicitud API se resuelve así. El UFS identifica al solicitante por la clave de acceso con la que se firmó la petición y comprueba que la firma sea válida y que la marca de tiempo esté dentro de un margen aceptable, lo que explica la exigencia de un reloj sincronizado. Después se reúnen las políticas que aplican al usuario: las propias, las de sus grupos y las de sus roles. La decisión sigue una regla simple: si alguna política niega de forma explícita la acción, se rechaza; si no hay negación y alguna política la permite, se acepta; si ninguna la menciona, se rechaza por defecto. Solo entonces la solicitud pasa al CLC.

E. Seguridad de la red y aislamiento

El aislamiento entre cargas de trabajo se logra en tres niveles. El primero es el hipervisor: cada instancia es una máquina virtual KVM [7] con su propio núcleo y memoria. El segundo son los grupos de seguridad, firewalls con estado que eucanetd convierte en reglas de iptables e ipsets en cada nodo; por defecto, un grupo bloquea el tráfico entrante y el usuario abre solo lo necesario. En la práctica, el grupo *default* quedó con los puertos 22 y 80 permitidos para que las instancias fueran alcanzables [6]. El tercer nivel es la red: en el modo EDGE el aislamiento recae en las reglas de los nodos, y en el modo VPCMIDO las redes virtuales aíslan el tráfico entre cuentas [3].

F. Cifrado y trazabilidad

El tráfico hacia la API y hacia la consola viaja cifrado por HTTPS: el manual indica que la consola solo responde por HTTPS en el puerto 443 y que la API se publica en *https://192.168.215.10:8773/* [6]. Los certificados son autofirmados, por lo que la confianza no está garantizada por una autoridad externa. La documentación consultada no describe un cifrado de los volúmenes o de los objetos en reposo activado por defecto, de modo que la protección de los datos almacenados recae en el cifrado del sistema de archivos del servidor o en el cifrado a nivel de aplicación.

En cuanto a la trazabilidad, cada componente deja registros en */var/log/eucalyptus* y las operaciones de la API quedan asociadas a la cuenta y al usuario que las firmó, lo que permite reconstruir quién lanzó o eliminó un recurso. Con todo, no se encontró en la práctica un servicio de auditoría centralizado equivalente a los de las nubes públicas.

G. Hallazgos de seguridad del laboratorio

Para que la instalación funcionara se tomaron decisiones que reducen la seguridad y que solo son aceptables en un laboratorio aislado. La Tabla XIII las resume junto con la medida recomendada.

TABLA XIII
RIESGOS DEL LABORATORIO Y MITIGACIÓN

Decisión en el laboratorio	Riesgo	Mitigación
SELinux en modo permisivo	Se pierde el control de acceso obligatorio	Políticas SELinux para Eucalyptus
firewalld desactivado	Todos los puertos expuestos en la VM	Abrir solo 22, 443, 8773-8775
Contraseña inicial del administrador	Acceso trivial a la nube	Cambiarla al terminar la instalación
Verificación de certificados desactivada	Ataques de intermediario	Certificado firmado por una CA
Python 2.7 y CentOS 7 sin soporte	Vulnerabilidades sin parche	Migrar a una plataforma con soporte

H. Buenas prácticas de endurecimiento

Las medidas de endurecimiento aplicables a esta arquitectura son las siguientes. Primero, cambiar las credenciales iniciales y crear usuarios con permisos mínimos en lugar de trabajar siempre con el administrador de la cuenta. Segundo, restringir los puertos de gestión con un firewall, en vez de desactivarlo. Tercero, volver a activar SELinux en modo *enforcing* en cuanto se disponga de las políticas necesarias, ya que el modo permisivo fue una concesión del laboratorio. Cuarto, instalar certificados válidos para la consola y la API. Quinto, mantener copias de seguridad de la base de datos de la nube y de las claves del servidor, porque su pérdida implica perder el control sobre todos los recursos.

I. Modelo de amenazas simplificado

Para ordenar los riesgos se pueden revisar los puntos de contacto de la arquitectura. La API pública (puerto 8773) es el principal punto de entrada y la superficie más expuesta; el riesgo es el robo de claves de acceso. Los puertos 8774 y 8775 solo deben ser alcanzables entre componentes, porque permiten ordenar la creación y destrucción de máquinas virtuales. El servicio de metadatos puede ser abusado desde una instancia comprometida para leer datos de arranque, por lo que no debe contener secretos. Por último, el servidor que aloja al CLC concentra las claves y la base de datos, de modo que debe protegerse como el activo más crítico.

Como ejemplo de control de red a nivel de usuario, el grupo de seguridad de una instancia se administra con comandos como los siguientes, que abren únicamente el puerto web a todo el mundo y el acceso SSH a una red de administración [3].

```
euca-authorize -P tcp -p 80 -s 0.0.0.0/0 default
euca-authorize -P tcp -p 22 -s 192.168.215.0/24 default
```

J. Resumen del modelo de seguridad

En conjunto, el modelo de seguridad de Eucalyptus descansa en tres ideas: identidad y permisos al estilo de IAM, aislamiento de las instancias mediante el hipervisor y los grupos de seguridad, y protección del plano de control mediante el control de los puertos y de las credenciales del servidor. Su punto más débil no está en el diseño sino en la operación: depende de que el administrador cambie las credenciales por defecto, cierre los puertos innecesarios y mantenga actualizado el sistema base, algo cada vez más difícil cuando el sistema base ya no tiene soporte.

K. Recomendaciones

Si Eucalyptus se usara fuera de un laboratorio, convendría aislar la red de gestión de la red de las instancias, restringir el acceso a los puertos 8773 a 8775 solo a los componentes que los necesitan, activar TLS con certificados válidos, rotar las claves de acceso, aplicar el principio de mínimo privilegio en las políticas IAM y mantener sincronizado el reloj de todos los servidores.

VII. CONCLUSIONES

A. Fortalezas y debilidades de la arquitectura

TABLA XIV
RESUMEN DE FORTALEZAS Y DEBILIDADES

Fortalezas	Debilidades
Arquitectura clara en tres niveles que facilita aprender cómo funciona una nube	Sin desarrollo activo: repositorio archivado en 2024
Compatibilidad con las API de AWS (EC2, S3, EBS, IAM)	Solo CentOS/RHEL 7 en la línea 4.4, sin soporte
Instalación Cloud-in-a-Box en una sola VM; en la práctica tardó 7 min 36 s	Instalador roto sin parches (Chef, EPEL, node.set)
Componentes modulares que pueden repartirse en varios servidores	Consola con errores por Python 2.7 y eventlet
Uso de tecnologías estándar: KVM, libvirt, iptables	Sin gestor de secretos y con certificados autofirmados

TABLA XV
COMPARACIÓN CONCEPTUAL CON OTROS ORQUESTADORES

Aspecto	Eucalyptus	OpenStack	Kubernetes
Unidad orquestada	Máquinas virtuales	Máquinas virtuales	Contenedores (pods)
API principal	Compatible con AWS	Propia de OpenStack	API de Kubernetes
Alcance	Nube de laboratorio	Nube a gran escala	Aplicaciones en contenedores
Estado del proyecto	Archivado en 2024	Activo	Activo

La comparación muestra que Eucalyptus ocupa el mismo nivel que OpenStack (infraestructura como servicio), mientras que Kubernetes orquesta una capa superior. Por eso los conceptos de nodo maestro, scheduler y controladores de Kubernetes tienen su equivalente aproximado en el CLC, el CC y los NC, pero la unidad que se planifica es una máquina virtual y no un pod.

En una valoración final, la arquitectura de Eucalyptus es clara, modular y fácil de seguir de punta a punta, y por eso resulta valiosa como herramienta de aprendizaje. Su principal limitación no es de diseño sino de mantenimiento: las dependencias que la rodean han avanzado y el proyecto no las ha seguido. Para un estudiante o un profesional que quiera comprender cómo se construye una nube IaaS, sigue siendo un caso de estudio completo; para quien necesite operar una nube hoy, conviene elegir una plataforma con soporte activo.

B. Recomendaciones de uso en un curso

Para un curso de tecnologías emergentes, Eucalyptus se puede usar como laboratorio guiado en tres etapas. En la primera, los estudiantes instalan la nube siguiendo un manual y registran los errores que encuentran. En la segunda, exploran la arquitectura con los comandos de verificación y relacionan cada servicio con el componente que lo ofrece. En la tercera, lanzan una instancia, la conectan a un volumen y la exponen con una dirección pública, para recorrer el ciclo completo de la sección IV. Es recomendable que cada estudiante trabaje con snapshots para poder repetir la práctica sin reinstalar el sistema base.

C. Casos de uso ideales

Eucalyptus es adecuado para tres escenarios: enseñanza de computación en la nube, donde permite ver y modificar cada componente de una nube real en una sola máquina; laboratorios de pruebas de compatibilidad con AWS, donde se necesita un entorno propio y desechable; y demostraciones de arquitectura IaaS. No es adecuado para producción por la falta de mantenimiento y de actualizaciones de seguridad.

D. Lecciones aprendidas de la práctica

1. Tomar un snapshot de la máquina virtual antes de instalar Eucalyptus permitió volver atrás en segundos tras los intentos fallidos [6].

2. Dimensionar bien el disco desde el inicio es crítico, porque imágenes, instancias y volúmenes comparten la partición donde está */var*.

3. Ejecutar el instalador dentro de una sesión de *screen* evita perder el proceso si se corta la conexión SSH, y la receta de limpieza *nuke* permite repetir una instalación a medias.

4. Los cambios de red en la fase 2 son delicados: el puente *br0* y la pérdida de los servidores DNS provocaron un fallo que no se detecta con un simple ping a una dirección IP.

5. Documentar cada error con su causa y su solución, como se hizo en el apartado de resolución de problemas del manual, ahorra tiempo a quien repita la instalación.

E. Conclusiones y trabajo futuro

1. Eucalyptus se organiza en tres niveles (nube, clúster y nodo), con el CLC como cerebro, el CC y el SC como administradores de clúster y el NC como ejecutor de máquinas virtuales sobre KVM.

2. La instalación Cloud-in-a-Box permitió comprobar esa arquitectura en una sola máquina virtual: los servicios aparecieron en estado *enabled*, la zona *default* con su clúster y las imágenes disponibles, y la consola web respondió por HTTPS [6].

3. Los flujos de lanzamiento de instancias, auto-escalado y failover siguen el mismo patrón de delegación jerárquica, con comunicación por servicios web y estados que reportan hacia arriba.

4. La seguridad se basa en IAM, en grupos de seguridad aplicados por eucanetd y en el aislamiento del hipervisor, pero la plataforma carece de un gestor de secretos y su pila está obsoleta, por lo que debe limitarse a entornos aislados.

5. Eucalyptus permite entender, con un esfuerzo de instalación razonable, cómo se organiza una nube IaaS: una capa de control única, capas de clúster que administran recursos y nodos que ejecutan la carga, todo coordinado por servicios web y por el reporte periódico de estados.

Trabajo futuro. Se proponen cuatro líneas de continuación. La primera es lanzar una instancia web sobre esta nube y documentar el despliegue de una aplicación, que es el objetivo del siguiente manual del grupo. La segunda es instalar y comparar la serie 5 [8] para medir qué dependencias obsoletas desaparecen. La tercera es comprobar el failover con dos servidores de gestión y un nodo adicional, lo que confirmaría de forma experimental el diagrama de la sección IV. La cuarta es comparar el rendimiento y la facilidad de operación de Eucalyptus con orquestadores actualmente mantenidos, como OpenStack, y con plataformas de contenedores.

TABLA XVI
LISTA DE ACRÓNIMOS

Sigla	Significado
CLC	Cloud Controller
UFS	User-Facing Services
OSG	Object Storage Gateway
CC	Cluster Controller
SC	Storage Controller
NC	Node Controller
EMI	Eucalyptus Machine Image
EBS	Elastic Block Store
IAM	Identity and Access Management
STS	Security Token Service
ELB	Elastic Load Balancer
KVM	Kernel-based Virtual Machine
IaaS	Infraestructura como servicio
RBAC	Control de acceso basado en roles

REFERENCIAS

[1] P. Mell y T. Grance, "The NIST definition of cloud computing," National Institute of Standards and Technology, Gaithersburg, MD, EE. UU., Spec. Publ. 800-145, sep. 2011. [En línea]. Disponible en: https://csrc.nist.gov/pubs/sp/800/145/final

[2] D. Nurmi et al., "The Eucalyptus open-source cloud-computing system," en *Proc. 9th IEEE/ACM Int. Symp. Cluster Computing and the Grid (CCGrid)*, Shanghái, China, may. 2009, pp. 124–131.

[3] Eucalyptus, "Eucalyptus 4.4.5 installation guide: Installation overview." Recuperado el 18 de octubre de 2026, de https://docs.eucalyptuscloud.org/eucalyptus/4.4.5/install-guide/index.html

[4] Eucalyptus, "Home · Eucalyptus wiki" [Repositorio archivado en agosto de 2024], GitHub. Recuperado el 18 de octubre de 2026, de https://github.com/eucalyptus/eucalyptus/wiki/Home

[5] Eucalyptus, "Eucalyptus cookbook" [Repositorio de software], GitHub. Recuperado el 18 de octubre de 2026, de https://github.com/eucalyptus/eucalyptus-cookbook

[6] W. Cueva, R. Vieira, J. Yugsi y J. Zambrano, "Manual de instalación: Eucalyptus – Cloud-in-a-Box sobre VMware," Grupo 3, Tecnologías Emergentes, Universidad de las Fuerzas Armadas ESPE, oct. 2026.

[7] A. Kivity, Y. Kamay, D. Laor, U. Lublin y A. Liguori, "kvm: the Linux virtual machine monitor," en *Proc. Linux Symposium*, Ottawa, Canadá, vol. 1, 2007, pp. 225–230.

[8] Eucalyptus, "Install Eucalyptus release packages," en *Eucalyptus 5 installation guide*. Recuperado el 18 de octubre de 2026, de https://docs.eucalyptus.cloud/eucalyptus/5/install_guide/eucalyptus/install_repos_intro/installing_euca_release/

[9] Cinc Project, "Cinc: Free-as-in-beer distribution of Chef Software's open source projects" [Sitio web]. Recuperado el 20 de octubre de 2026, de https://cinc.sh/

[10] Fedora Project, "EPEL 7 archive" [Repositorio de paquetes]. Recuperado el 20 de octubre de 2026, de https://archives.fedoraproject.org/pub/archive/epel/7/

[11] The CentOS Project, "CentOS Vault mirror: 7.9.2009/isos/x86_64." Recuperado el 18 de octubre de 2026, de https://vault.centos.org/7.9.2009/isos/x86_64/